Solution: Clone Graph

Problem Statement

Given a reference of a node in a connected undirected graph, return a deep copy (clone) of the graph. Each node in the graph contains a value (`int`) and a list (`List[Node]`) of its neighbors.

Example 1:

Input:

```
1--2
|  |
4--3
```

Expected Output:

```
1--2
|  |
4--3
```

Explanation: The graph has four nodes with the following connections:

- Node `1` is connected to nodes `2` and `4`.
- Node `2` is connected to nodes `1` and `3`.
- Node `3` is connected to nodes `2` and `4`.
- Node `4` is connected to nodes `1` and `3`.

Example 2:

Input:

```
   1--2
  /    \
 5      3
        |
        4
```

Expected Output:

```
   1--2
  /    \
 5      3
        |
        4
```

Explanation: The graph consists of five nodes with these connections:

- Node `1` is connected to nodes `2` and `5`.
- Node `2` is connected to nodes `1` and `3`.
- Node `3` is connected to nodes `2` and `4`.
- Node `4` is connected to node `3`.
- Node `5` is connected to node `1`.

Example 3:

Input:

```
   1--2
  /    \
 4      3
  \    /
   5--6
```

Expected Output:

```
   1--2
  /    \
 4      3
  \    /
   5--6
```

Explanation: The graph has six nodes with the following connections:

- Node `1` is connected to nodes `2` and `4`.
- Node `2` is connected to nodes `1` and `3`.
- Node `3` is connected to nodes `2` and `6`.
- Node `4` is connected to nodes `1` and `5`.
- Node `5` is connected to nodes `4` and `6`.
- Node `6` is connected to nodes `3` and `5`.

Solution

To deep clone a given graph, the primary approach is to traverse the graph using Depth-First Search (DFS) and simultaneously create clones of the visited nodes. A hashmap (or dictionary) is utilized to track and associate original nodes with their respective clones, ensuring no duplications.

1. **Initialization**: Create an empty hashmap to match the original nodes to their clones.
2. **DFS Traversal and Cloning**: Traverse the graph with DFS. When encountering a node not in the hashmap, create its clone and map them in the hashmap. Recursively apply DFS for each of the node's neighbors. After cloning a node and all its neighbors, associate the cloned node with the clones of its neighbors.
3. **Termination**: Once DFS covers all nodes, return the cloned version of the starting node.

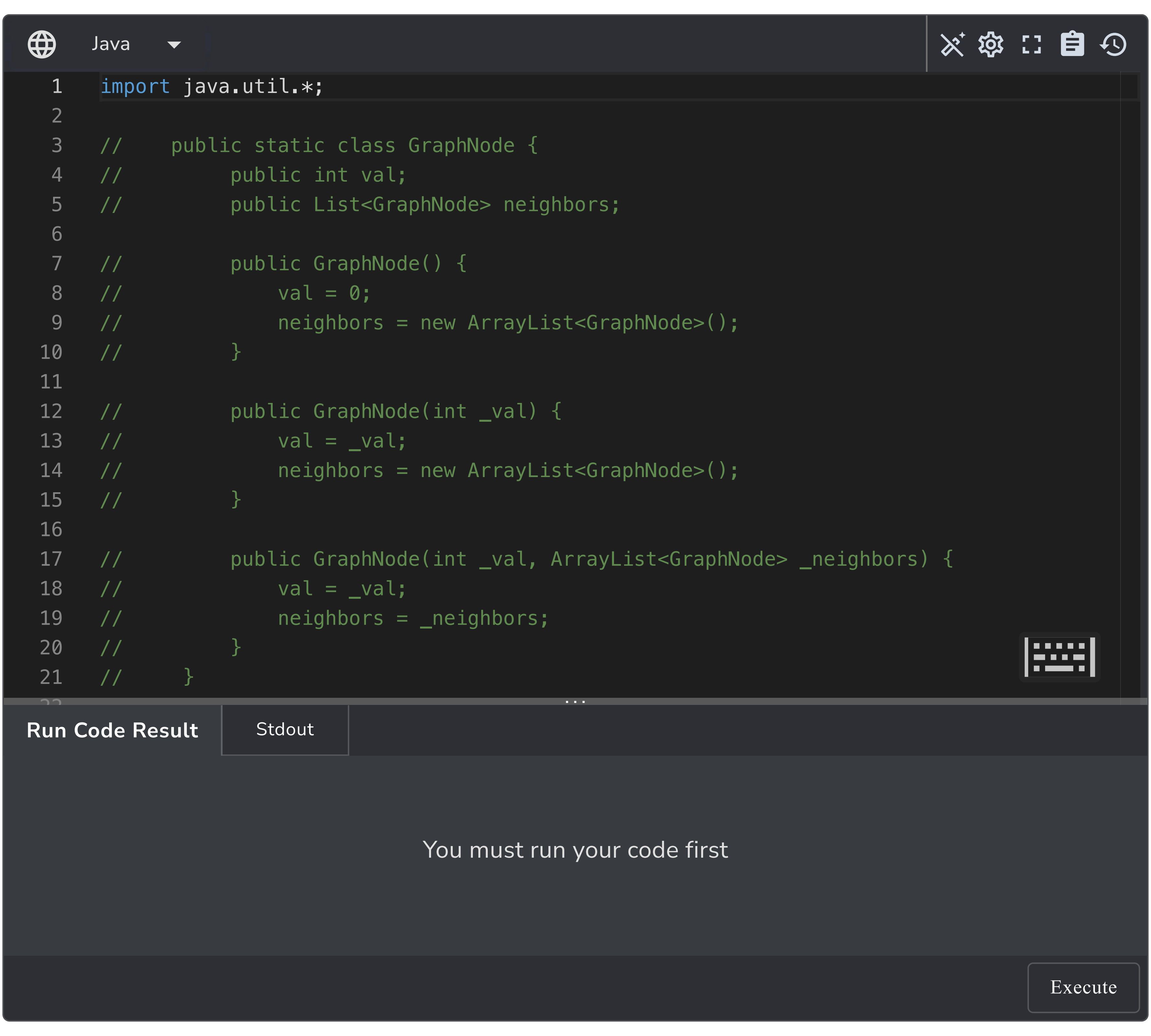
Algorithm Walkthrough (using Example 1):

For the input graph:

```
1--2
|  |
4--3
```

- Start with an empty hashmap `visited`.
- Begin DFS with node `1`.
  - Node `1` isn't in `visited`. Clone it to get `1'` and map `(1, 1')` in the hashmap.
  - For each neighbor of node `1`, apply DFS.
    - First with `2`.
      - Node `2` isn't in `visited`. Clone to get `2'` and map `(2, 2')`.
      - Node `2`'s neighbors are `1` and `3`. Node `1` is visited, so link `2'` to `1'`. Move to `3`.
        - Node `3` isn't in `visited`. Clone to get `3'` and map `(3, 3')`.
        - Node `3` has neighbors `2` and `4`. Node `2` is visited, so link `3'` to `2'`. Move to `4` .
          - Node `4` isn't in `visited`. Clone to get `4'` and map `(4, 4')`.
          - Node `4` has neighbors `1` and `3`, both visited. Link `4'` to `1'` and `3'`.
- With DFS complete, return the clone of the starting node, `1'`.

Code



```java
import java.util.*;

//    public static class GraphNode {
//          public int val;
//          public List<GraphNode> neighbors;

//          public GraphNode() {
//              val = 0;
//              neighbors = new ArrayList<GraphNode>();
//          }

//          public GraphNode(int _val) {
//              val = _val;
//              neighbors = new ArrayList<GraphNode>();
//          }

//          public GraphNode(int _val, ArrayList<GraphNode> _neighbors) {
//              val = _val;
//              neighbors = _neighbors;
//          }
//      }
```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(N+M) where N is the number of nodes and M is the number of edges. Each node and edge is visited once.
- **Space Complexity**: O(N) as we are creating a clone for each node. Additionally, the recursion stack might use O(H) where H is the depth of the graph (in the worst case this would be O(N)).